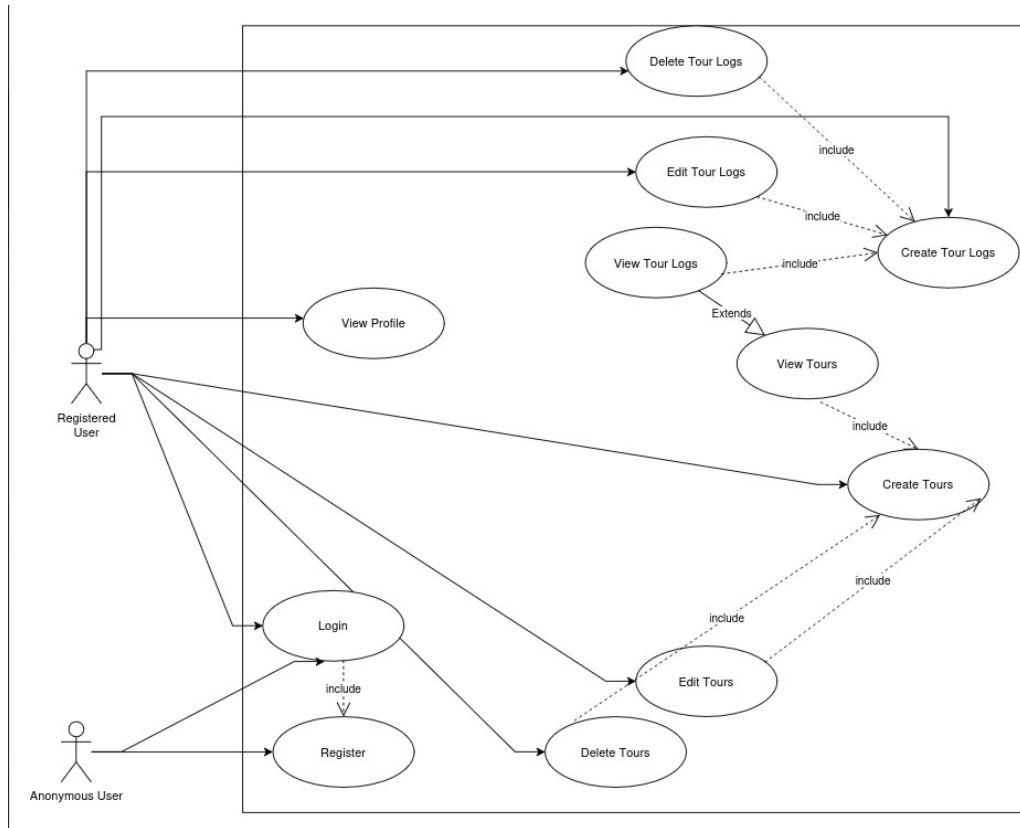


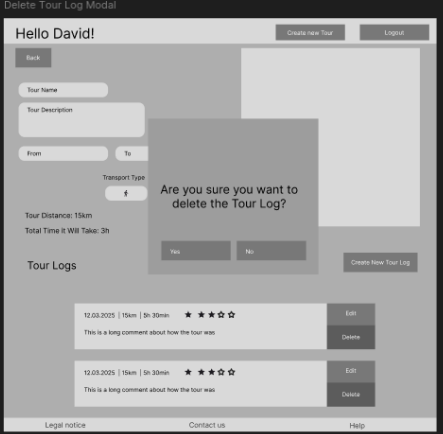
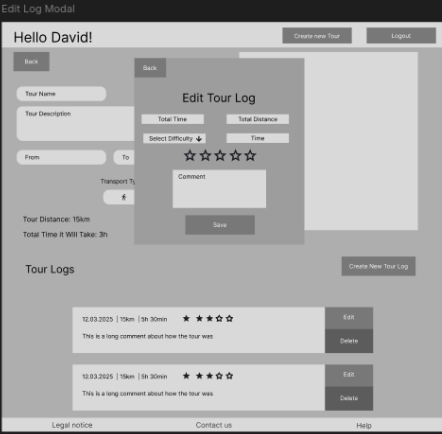
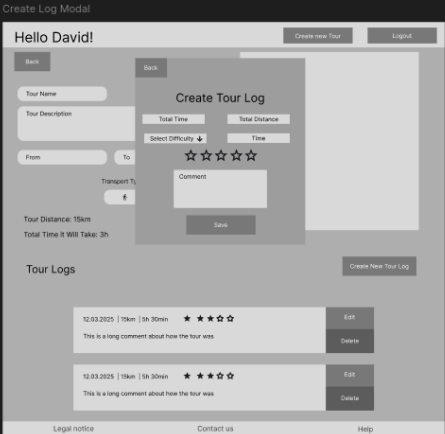
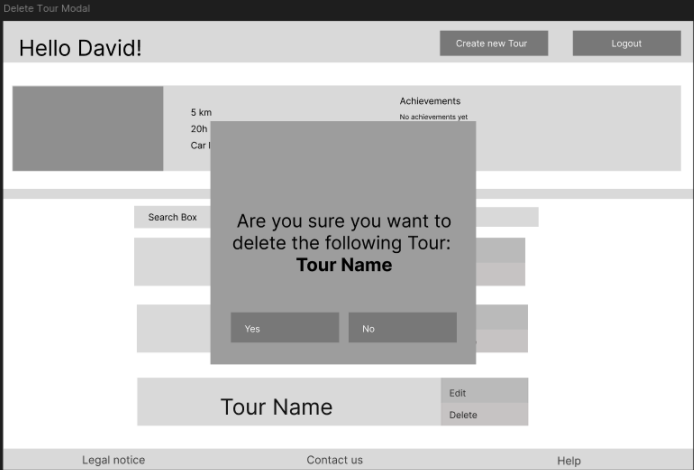
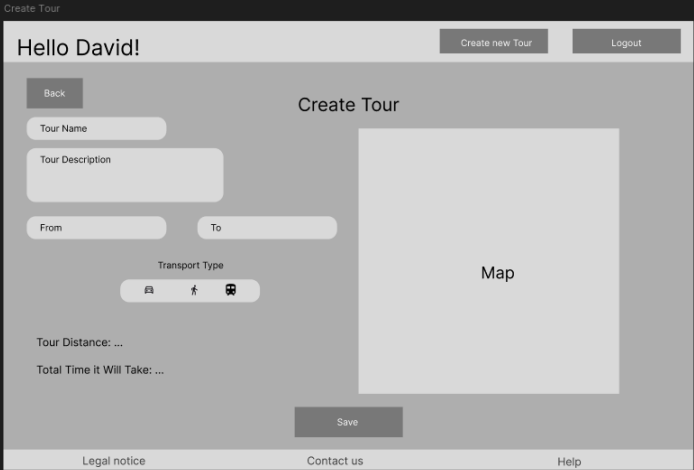
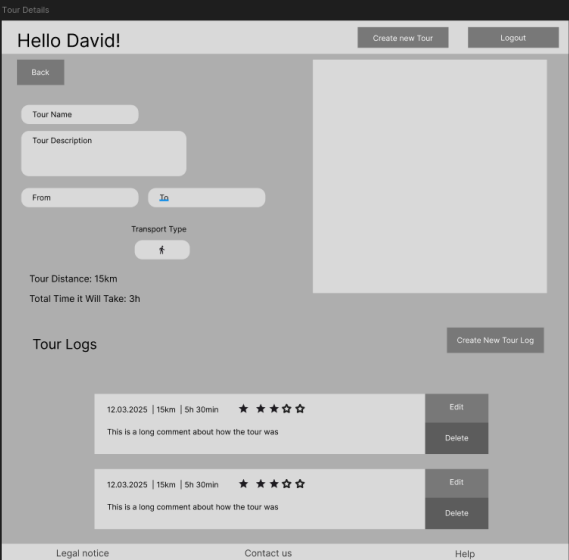
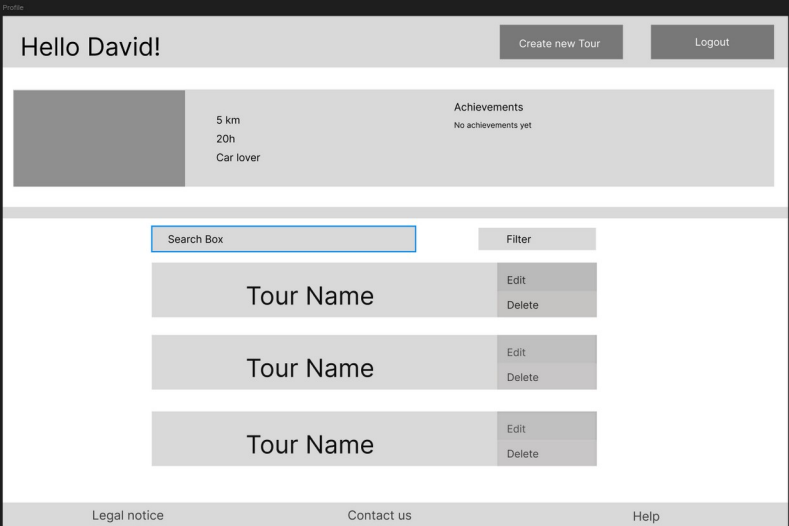
Tour Planner Protocol

Olesja Nickel, David Nawloka

Git Link: https://github.com/lesjanikel/Tour_Planner.git

Usecase diagram:





UX Description

We tried to keep the UX as simple and easy to understand as possible. To access most features one has to register and login. After that one has access to the tour dashboard, that displays some stats about the user (total distance/time in all tours, favorite transportation method) and some achievements the user has gotten. In addition to that, a logged in user can create a new tour by adding a tour name, description a start-/end location and a transport type. He then sees the tour in his dashboard. Clicking on the tour, takes the user to the tour detail view. There he can add tour logs - documenting his journey on the tour, including date, difficulty, distance, time and rating.

Application Architecture

Layer Structure

The application follows a strict three-layer architecture:

- Presentation Layer: Angular 21 SPA with standalone components, Angular Signals for state management, and reactive forms.
- Business Layer (Spring Boot Services): TourService, TourLogService, AuthService, GeocodeService, ImageStorageService - all business rules enforced here.
- Data Access Layer (Spring Data JPA Repositories): TourRepository, TourLogRepository, UserRepository backed by PostgreSQL.

Backend Class Overview

Controllers (REST endpoints):

- TourController - CRUD for tours, search, import/export
- TourLogController - CRUD for tour logs
- TourImageController - image upload and retrieval
- GeocodeController - address autocomplete proxy
- SessionController - login and register
- UserController - profile information

Models:

- User - id, username, passwordHash; owns tours
- Tour - id, name, description, from/to coordinates, transportType, distanceKm, durationSec, routeGeoJson, imageUrl, owner, logs
- TourLog - id, dateTime, comment, difficulty, totalDistance, totalTime, rating, tour

Security:

JwtService generates and validates HS256 JWT tokens. JwtAuthenticationFilter intercepts every request, extracts the token from the Authorization header and sets the Spring Security context.

Frontend Structure

Services:

- TourService - holds tours signal, calls backend CRUD and search endpoints
- TourLogService - holds logs signal, calls backend log endpoints
- AuthService - holds user signal, stores JWT in localStorage
- GeocodeService - calls ORS geocode autocomplete
- MapFacadeService - facade over Leaflet to keep map concerns out of components
- ToastService - manages notification toasts via a signal
- AchievementsService - pure client-side logic computing achievement badges from tour data

Reusable component: AddressAutocomplete - a custom input component with debounced geocoding suggestions, emitting a structured GeocodeFeature on selection. Used in both TourForm (from/to fields).

Design Patterns Used

- Facade (MapFacadeService): hides Leaflet API complexity from components; components call initMap(), setRoute(), destroyMap() only.
- Repository (Spring Data JPA): TourRepository, TourLogRepository, UserRepository abstract all database access.
- Interceptor (Angular HTTP): AuthInterceptor attaches the JWT Bearer token to every outgoing request. ErrorInterceptor converts HTTP error responses to typed errors.
- MVVM (Angular): components expose signals as the ViewModel; templates bind directly to computed signals without additional logic.

Sequence Diagram: Full-Text Search

The following describes the flow when a user types into the search box on the tour list page:

1. User types in search input - TourList component calls TourService.search(query).
2. TourService sends GET /api/tours/search?q={query} with JWT Authorization header.
3. JwtAuthenticationFilter validates the token and sets the authenticated User in the Spring Security context.
4. TourController.search() receives the request, reads the authenticated principal.
5. TourController delegates to TourService.search(query, user).
6. TourService calls TourRepository.search(ownerId, query).
7. TourRepository executes a JPQL query: searches LOWER(name), LOWER(description) and LOWER(log.comment) using LIKE %query%, filtered by ownerId.
8. Matching Tour entities are mapped to TourDto and returned as JSON.
9. TourService.search() updates the tours signal with the result.
10. Angular's computed signal filteredTours() re-evaluates and the tour list re-renders.

The search is case-insensitive and spans tour name, description, and all associated tour log comments in a single database query using a LEFT JOIN.

Unit Tests: Selection and Rationale

Tests were written for classes with non-trivial logic where a mistake would be silent or hard to detect at runtime. Static pages and pure HTTP-delegation classes (no branching logic) were excluded.

Backend

- **JwtServiceTest:** The `JwtService` is responsible for generating and verifying authentication tokens, which are central to the security of the entire application. A unit test confirms that a generated token correctly embeds the username as its subject, verifying the fundamental token generation behaviour.
- **AuthServiceTest:** The `AuthService` enforces the registration and login rules that control access to the system. Unit tests verify that duplicate usernames are rejected before any persistence occurs and that valid credentials produce a correctly issued token, protecting against unauthorized account creation and access. Correct error throwing is also tested.
- **TourServiceTest / TourLogServiceTest:** Both services enforce owner-scoped data access by querying only resources that belong to the authenticated user. Unit tests verify that requests for non-existent or unowned resources throw the expected exception, ensuring that the access boundary between users cannot be silently bypassed.
- **GeocodeServiceTest:** The `GeocodeService` delegates queries to the external `OpenRouteService` API, which has usage limits. Unit tests confirm that blank or empty input does not trigger an API call, preventing unnecessary quota consumption and potential runtime errors from the external service.
- **GlobalExceptionHandlerTest:** The `GlobalExceptionHandler` translates application exceptions into HTTP responses consumed by the frontend. Unit tests verify that the most critical mappings are correct: `NotFoundException` produces a 404 response and `BadCredentialsException` produces a 401 response, ensuring that the frontend receives consistent and meaningful error information.
- **ImageStorageServiceTest:** The `ImageStorageService` performs file system operations on user-uploaded content with content-type validation. Unit tests confirm that unsupported file types are rejected before any file system interaction occurs and that valid uploads are stored correctly, preventing storage corruption.

Frontend

- **AchievementsService:** The `AchievementsService` contains pure business logic with no external dependencies, making it well-suited for isolated unit testing. The correctness of achievement evaluation relies on correctly aggregating data across all tours, which makes automated verification of the accumulation logic essential.
- **AuthService / Guards:** The `AuthService` manages the authenticated user session, and the route guards rely on its state to protect restricted routes. Unit tests verify that the session correctly reflects authentication status and that route guards block access for unauthenticated users.
- **TourList computed signals:** The `filteredTours` and `totalDistance` values are computed signals derived from the tour data and active filters. A regression in this logic would cause incorrect data to be displayed without any visible error, making automated verification of the derivation logic important.
- **Form validation (TourForm, TourLogForm, Register, Login):** Input validation in the tour and log forms as well as in the registration and login flows prevents malformed data from being submitted to the backend. Unit tests confirm that the defined validation constraints are correctly enforced by the form controls.
- **TourLogService (HTTP):** The `TourLogService` maintains an in-memory signal that must remain consistent with the server state after each operation. Unit tests using `HttpTestingController` verify that delete and update operations correctly synchronise the local signal with the server response.
- **ToastService / AddressAutocomplete:** `ToastService` and `AddressAutocomplete` are shared components used across multiple features of the application. Unit tests ensure their core behaviour remains stable, as any regression would have a broad impact on the user experience.

Time Tracking

- Use case diagram, wireframes / UI prototype: ~6h
- Architecture planning: ~2h
- Use case diagram, wireframes and UI prototype: ~3h
- Project setup, Docker, database schema, Spring Boot skeleton: ~7h
- Authentication (JWT, register, login, guards): ~6h
- Logging setup (SLF4J / log4j, adding log statements throughout the application): ~2h
- Tour CRUD backend + frontend: ~8h
- Tour Log CRUD backend + frontend: ~4h
- OpenRouteService integration (routing + geocoding): ~4h
- Leaflet map integration (MapFacadeService): ~2h
- Search, computed attributes (popularity, child-friendliness): ~4h
- Import / Export feature: ~4h
- Image upload feature: ~3h
- Achievements system (unique feature): ~2h
- Logging (log4j/SLF4J setup + adding log statements): ~2h
- Unit tests (frontend + backend): ~4h
- Styling, UX polish, bug fixes: ~5h
- Documentation: ~2h
- Total: ~65h